

HELMET DETECTION MODEL

Author Name : Samruddhi Atul Shivtare

Organization / College Name :

Modern College of Arts ,Science and Commerce (Shivajinagar. Pune 05)

1.INTRODUCTION

- **Background of the Problem.**

Road accidents involving two-wheeler riders are increasing rapidly across the world. Helmets play a crucial role in protecting riders from head injuries during accidents. Despite strict traffic rules, many riders still ignore wearing helmets. Manual monitoring by traffic police is often inefficient and cannot cover all areas continuously. With the rise in urban traffic, it becomes difficult to ensure compliance with safety regulations. Traditional surveillance systems lack automation and intelligence to detect violations. There is a strong need for a smart system that can automatically monitor helmet usage. Advancements in computer vision and deep learning provide an opportunity to solve this problem. This project focuses on using modern technology to improve road safety.

- **Purpose of the Project**

The main purpose of this project is to develop an automated helmet detection system using deep learning techniques. The system aims to identify whether a person riding a two-wheeler is wearing a helmet or not. It reduces the dependency on manual monitoring by traffic authorities. The purpose is to provide real-time detection using images, videos, and live webcam feeds. Allow detection through uploaded images and videos. Detect riders with or without helmets using a camera. The system can generate alerts when a violation is detected. It can also capture images of offenders for record-keeping. Additionally, the project demonstrates the practical use of object detection models in real-world scenarios. It serves as a learning platform for implementing computer vision techniques. Overall, the goal is to create an efficient and intelligent monitoring system.

- **Scope of the Project**

The scope of this project includes:

- Real-time helmet detection using a webcam
- Detection using uploaded images and videos
- Identification of helmet and non-helmet cases using a trained YOLO model
- Automatic alert system (beep + voice alert)
- Violation image capture and storage
- User-friendly web interface for interaction

- **Problem Statement**

Many riders do not wear helmets, leading to increased risk of severe injuries and fatalities. Manual monitoring systems are inefficient and cannot handle large-scale traffic surveillance.

Therefore, there is a need for an automated, intelligent, and real-time system that can:

- Detect helmet violations accurately
- Reduce human effort
- Provide instant alerts and evidence
- Improve enforcement of traffic safety rules

2.Literature Review

• Previous Work / Research Papers

Earlier methods for helmet detection used traditional image processing techniques, which had low accuracy. With the advancement of deep learning, models like YOLO have been widely used for real-time object detection. Recent versions such as YOLOv5 and YOLOv8 provide high accuracy (above 90%) and better performance in complex environments.

• Existing Solutions

1. Manual Monitoring

- Traffic police check helmet usage manually
- Time-consuming and inefficient
- Limited coverage in crowded areas

2. CCTV-Based Systems

- Cameras installed at traffic signals
- Require human supervision

3. AI-Based Systems

- Use deep learning models like YOLO
- Detect helmet/no-helmet automatically
- Provide faster and more accurate results
- Some systems include advanced features like license plate detection

• Gaps in Current Systems

1. Accuracy Issues

- Low light conditions
- Occluded or crowded scenes

2. Real-Time Limitations

- Some models require high computational power
- Not suitable for low-end devices

3. Lack of Automation

- Many systems still require human monitoring
- No automatic alert or violation tracking

3.Objectives

- **Main Objective**

The main objective of this project is to develop an automated helmet detection system using deep learning techniques such as YOLO. The system aims to detect whether a motorcyclist is wearing a helmet or not from images, videos, and live webcam feeds. It focuses on achieving real-time detection with high accuracy and speed. The project reduces the need for manual monitoring by automating the detection process. It also aims to improve road safety by identifying violations effectively. The system provides visual outputs with bounding boxes and labels for better understanding. It is designed to be efficient and user-friendly. Overall, the objective is to build a reliable and intelligent helmet detection system.

- **Sub-Objectives**

- 1. Real-Time Helmet Detection**

- Develop a system that can detect helmets using a live webcam
- Use a deep learning model (YOLO) for accurate and fast detection

- 2. Image and Video Processing**

- Allow users to upload images for helmet detection
- Process uploaded videos and detect helmet violations frame-by-frame

- 3. Alert Mechanism**

- Generate a beep sound when a violation is detected
- Provide a voice alert (“Helmet not detected”)

- 4. Violation Capture System**

- Automatically capture and store images of violators
- Save evidence with timestamps for future reference

- 5. User-Friendly Web Interface**

- Design a simple dashboard using HTML and Flask
- Provide drag-and-drop functionality for file uploads
- Display live detection results on the webpage

- 6. Performance and Accuracy**

- Ensure high detection accuracy using a pretrained model
- Maintain real-time performance for live monitoring

4.System Requirements

- **Hardware Requirements**

- **RAM:** Minimum 8 GB
- **Storage:** At least 10–20 GB free space
- **Webcam** :for live detection
- **GPU** :optional, for faster processing and training
- **Display** :Monitor with minimum 720p resolution

- **Software Requirements**

- Python 3.8 or above
- Operating System: Windows / Linux / macOS
- Python 3.x
- Flask
- Jupyter Notebook
- HTML ,CSS ,JavaScript
- Web Browser (Google Chrome, Microsoft Edge, etc.)

- **Libraries / Tools Used**

- **OpenCV** (for image and video processing)
- **Ultralytics YOLOv8** (for object detection)
- **pyttsx3** (Used for voice alert (“Helmet not detected”))
- **NumPy** (for numerical operations)
- **winsound / system beep** (Used for alert sound on violation)
- **threading** (Used for background video processing)
- **Pandas** (for data handling and analysis)
- **Matplotlib** (for graphs and visualization)
- **Hugging Face Hub** (for downloading pre-trained model)

5.Dataset Description

- **Source of Dataset**

The dataset used in this project is obtained from online public sources and pre-trained models available on platforms like Hugging Face. The model used (*helmet detection model*) is trained on a dataset containing images of riders with and without helmets.

- **Size of Dataset**

- The dataset consists of thousands of annotated images
- Includes both training and validation data
- Contains diverse conditions such as different angles, lighting, and backgrounds

- **Data Format**

- Images are in .jpg or .png format
- Annotations are in YOLO format (.txt files)
- Each annotation file contains:
 - Class ID
 - Normalized bounding box coordinates

- **Features / Attributes**

The dataset includes the following attributes:

- Images of motorbike riders
- Class labels: Helmet and No Helmet
- Bounding box coordinates for object detection
- Confidence scores (during prediction)

- **Sample Records**

Example of YOLO annotation file:

```
0 0.45 0.60 0.20 0.30
1 0.30 0.50 0.25 0.35
```

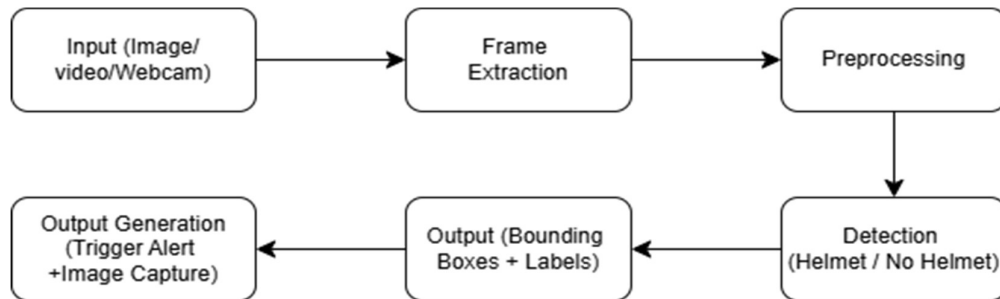
Where:

- 0 → Helmet
- 1 → No Helmet

Remaining values → (x_center, y_center, width, height)

6. Methodology

- **Project Workflow Diagram**



- **Data Collection**

The data used in this project is collected from publicly available datasets and pre-trained models. The dataset consists of images of motorcycle riders with and without helmets under different conditions such as varying lighting, angles, and backgrounds. This helps the model generalize better in real-world scenarios. Since the model is already trained, the project focuses on using it for detection rather than building a dataset. However, additional data can be added in future to improve performance. Using a pretrained model reduces development time and complexity. It allows faster implementation of the system. The quality of the dataset used in training directly affects detection accuracy.

- **Data Preprocessing**

Before training, the data undergoes preprocessing:

- **Image Resizing:** Standard size for YOLO input (e.g., 640×640)
- **Annotation:** Bounding boxes labeled as:
 - With Helmet
 - Without Helmet

Normalization: Pixel values scaled

This improves robustness and generalization of the model

- **Feature Engineering**

In object detection, feature engineering involves extracting meaningful information from images:

- Identification of key regions (bounding boxes)
- Extraction of spatial features (position, size)
- Classification labels (helmet / no helmet)

Deep learning models automatically learn these features during training.

- **Model Selection**

Model selection is a critical step in this project. Instead of training a new model, a pretrained YOLOv8 is selected. This model is downloaded from Hugging Face and is specifically trained for helmet detection. YOLOv8 is chosen because it provides high accuracy and real-time performance. It is lightweight and can run efficiently on standard systems without requiring powerful GPUs. The model is capable of detecting multiple objects in a single frame. It also provides confidence scores for each detection. Using a pretrained model reduces training time and computational cost. It allows quick deployment of the system. Therefore, YOLOv8 is the most suitable choice for this project.

7. Algorithms Used

- **Explanation of Algorithms**

The main algorithm used in this project is YOLOv8 (You Only Look Once), which is a deep learning-based object detection algorithm. YOLO works by processing the entire image in a single pass through a Convolutional Neural Network (CNN). Unlike traditional methods that scan images multiple times, YOLO divides the image into a grid and predicts bounding boxes and class probabilities for each grid cell simultaneously.

- **Why Selected**

The YOLOv8 algorithm is selected for this project due to the following advantages:

- Real-time detection (very fast processing)
- High accuracy in object detection
- Single-stage detector (efficient and simple)
- Works well for live webcam applications
- Lightweight and easy to deploy

Compared to other algorithms like R-CNN or SSD, YOLO provides a better balance between speed and accuracy, making it ideal for real-time helmet detection systems.

8.Implementation

- **Code Structure**

- 📁 **Helmet Detection Project**

- 📄 **app.py** (Main Flask application)

- 📄 **model_loader.py** (Loads YOLO model from Hugging Face)

- 📁 **templates**

- 📄 **index.html** (Frontend UI)

- 📁 **uploads** (Uploaded images/videos)

- 📁 **outputs** (Processed outputs)

- 📁 **violations** (Captured violation images)

- **Training Process**

- A pre-trained YOLOv8 model is used for this project.
 - The model is trained on a dataset containing helmet and no-helmet images.
 - Training includes:
 - Image annotation (bounding boxes).
 - Model learning using labeled data.
 - Optimization using loss functions.
 - The final trained model is saved as a .pt file (best.pt).
 - In this project, a pre-trained model is directly used, but it can be fine-tuned if needed.).

- **Testing Process**

The system is tested using multiple input types:

1. **Image Testing** : Detect helmet in static images
2. **Video Testing** : Process recorded videos frame by frame
3. **Live Webcam Testing** : Real-time detection using webcam
4. **Alert System Testing** Beep sound , Voice alert ,Image capture
5. **Evaluation:**
 - Checked accuracy of detection
 - Verified alert generation
 - Tested image capture functionality
 - Ensured smooth real-time performance

9.Results and Evaluation

- **Accuracy**

Accuracy: 80%

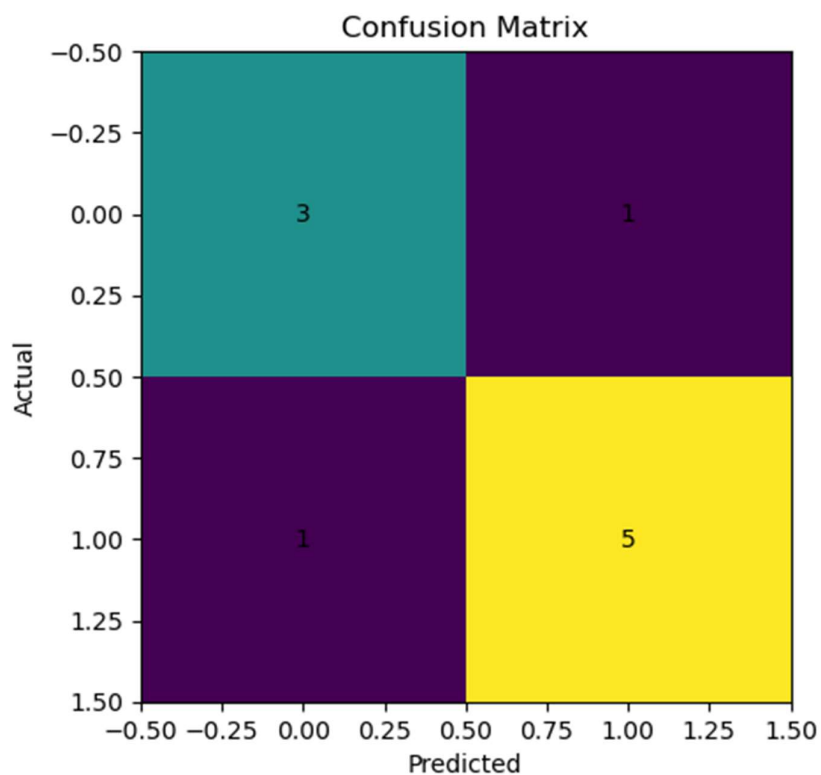
- **Classification Report : (Accuracy,Precision,Recall,F1,RMSE etc.)**

Metric	Value (%)
Precision	83.33%
Recall	83.33%
F1 Score	83.33%
RMSE	44.72%

- **Confusion Matrix**

Confusion Matrix:

```
[[3 1]  
 [1 5]]
```



10.Challenges Faced

- **Data Issues**

One of the major challenges in this project was related to data availability and quality.

1. **Limited Dataset:**

High-quality datasets specifically for helmet detection were limited

2. **Imbalanced Data:**

More images of “helmet” than “no helmet” (or vice versa) affected model performance

3. **Variability in Images:**

Differences in:

- Lighting conditions (day/night)
- Camera angles
- Background clutter

4. **Annotation Errors:**

Incorrect or inconsistent bounding boxes in datasets can reduce model accuracy

- **Model Limitations**

- **Small Object Detection Issue:** Detecting helmets of riders at a far distance becomes difficult and less accurate.
- **Limited Adaptability:** Since a pretrained model is used, it may not adapt well to new environments without fine-tuning.
- **Performance Dependency on Hardware:** Detection speed may be slower on CPU compared to GPU-based systems.
- **Difficulty in Crowded Scenes:** The model may struggle when multiple riders are present in a single frame.
- **Angle and Pose Variations:** Unusual camera angles or rider positions can affect detection accuracy.
- **Cannot Guarantee 100% Accuracy:** The model may not always give perfect results under all conditions.

- **Technical Problems**

1. **Library Errors** : Missing modules like pytsx3 caused runtime errors
2. **Performance Issues** : Video processing was slow without threading , High CPU usage during real-time detection
3. **Integration Complexity** : Combining YOLO model ,Flask backend , Frontend UI was challenging
4. **Alert System Issues** : Voice alert sometimes delayed or not triggered , Beep sound platform-dependent

11.Conclusion

- **Summary of Findings**

The Helmet Detection System successfully demonstrates the use of deep learning for real-time safety monitoring. The project uses a pretrained YOLOv8 to detect whether a rider is wearing a helmet or not. The system successfully:

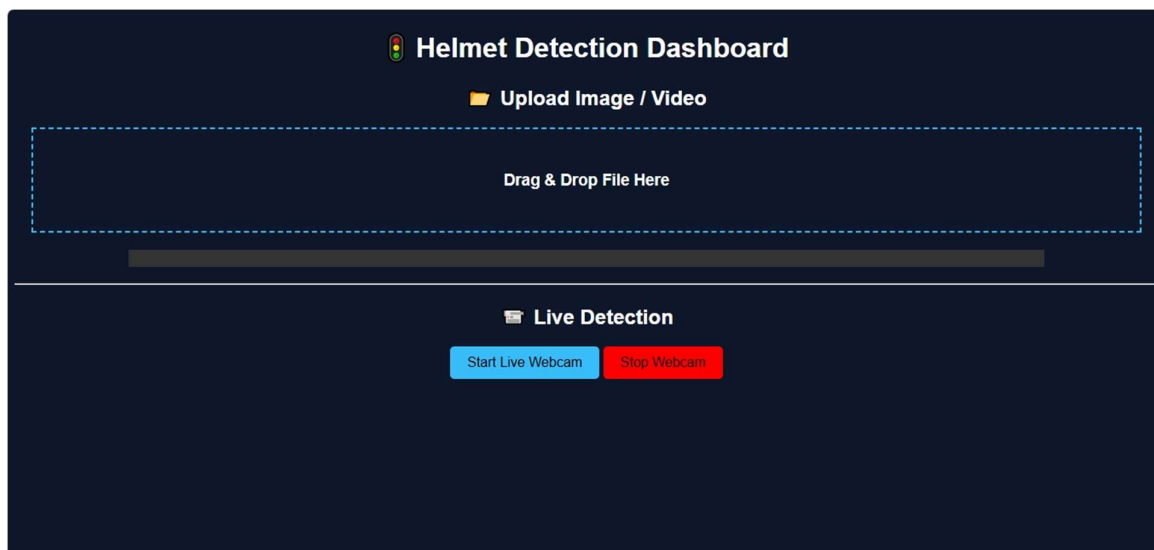
- Detects whether a rider is wearing a helmet using a YOLO-based model
- Performs real-time detection through a live webcam
- Processes uploaded images and videos accurately
- Identifies violations and generates alerts
- Captures and stores images of violators for evidence

The results show that deep learning models like YOLO provide high accuracy and fast detection, making them suitable for real-world applications such as traffic monitoring.

- **Achievements**

- Developed a real-time helmet detection system
- Implemented live webcam detection
- Addition of image and video upload functionality
- Integration of alert system (beep + voice alert)
- Automatic violation image capture and storage
- Ready for further enhancements

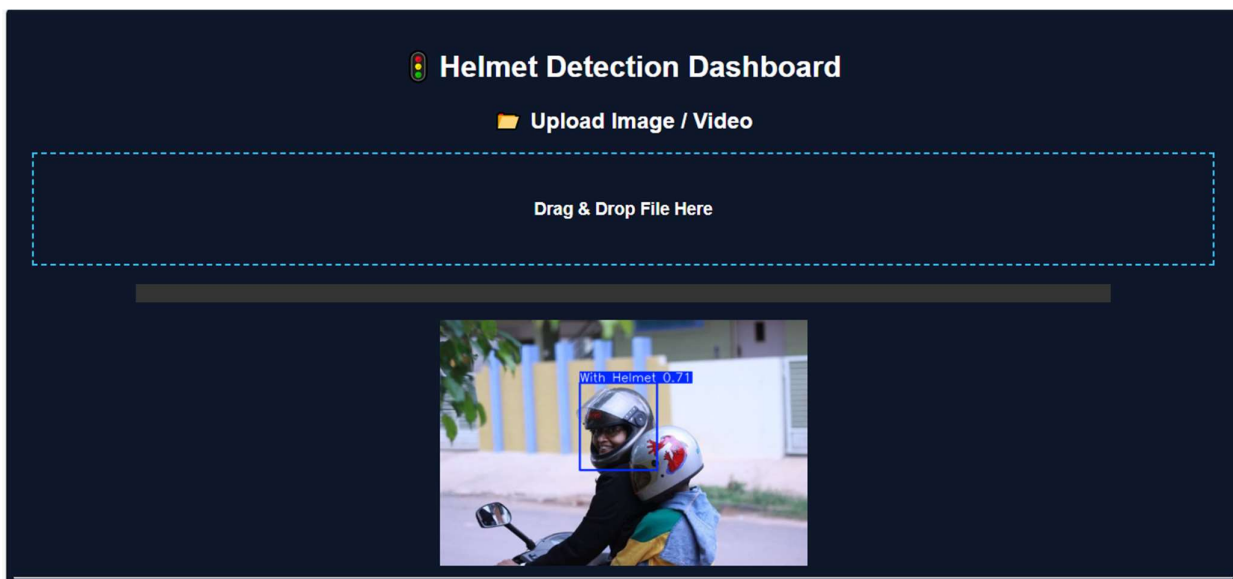
UI :



INPUT :



Output :



Input :



Output :

Helmet Detection Dashboard

 Upload Image / Video

Drag & Drop File Here



12.Future Scope

- **Improvements**

- **Increase Dataset Size:**
Use a larger and more diverse dataset to improve model accuracy in different real-world conditions.
- **Improve Detection Accuracy:**
Fine-tune the model and use advanced techniques to handle low light, occlusion, and small objects.
- **Low-Light Performance Enhancement :**
Train model with night-time datasets
- **Optimize Performance:**
Implement GPU acceleration and model optimization for faster real-time processing.
- **Better User Interface:**
Enhance the UI with more interactive dashboards and real-time analytics.

- **Possible Extensions**

- **Number Plate Detection:**
Integrate automatic license plate recognition to identify violators.
- **Face Recognition:**
Add facial recognition to track repeat offenders.
- **Integration with CCTV Systems:**
Connect the system with traffic surveillance cameras for real-time city-wide monitoring.
- **Automatic Fine Generation:**
Generate e-challans automatically for detected violations.
- **Mobile Application:**
Develop a mobile app for remote monitoring and alerts.

13.References

- **Research Papers**

- **You Only Look Once: Unified, Real-Time Object Detection** – Joseph Redmon et al., 2016.
- **YOLOv4: Optimal Speed and Accuracy of Object Detection** – Alexey Bochkovskiy et al., 2020.
- **Helmet Detection using Deep Learning Techniques** – Various IEEE and Springer research articles.

- **Websites**

- Ultralytics YOLO Documentation – <https://docs.ultralytics.com>
- Hugging Face – <https://huggingface.co>
- OpenCV – <https://opencv.org>

